ESSENTIALS OF ARTIFICIAL INTELLIGENCE
INDIVIDUAL CONTRIBUTION SHEET

Student Name	SNMRDS Arambepola	**Registration No.**	D/BIT/24/0039
Degree / Intake	BIT / 41	**Group No.**	40
Project Title	AI-Powered Dynamic Public Transport Fleet Optimizer (Route 100)		

Technical Contributions:

Task / Component	My Contribution	Tools / AI Technique Used	Evidence / Output
Dynamic Dispatch Headway Logic	Developed the fleet dispatch algorithm in calculate_dispatch(): determined bottleneck demand (peak = max(predictions)), calculated fleet requirements based on 80pax capacity (buses = max(1, ceil(peak / 80))), and clamped headway between 3 and 30 minutes (headway = clamp(round(60 / buses), 3, 30)) to mitigate bus bunching.	Python, Rule-Based Optimization, Fleet Sizing	backend/dispatch.py (calculate_dispatch); Slide Deck Slide 8
Congestion-Aware Corridor ETA Model	Modeled realistic end-to-end corridor travel times along the 8 Galle Road segments ([14, 12, 10, 12, 8, 8, 10] min). Applied a 1.25x peak congestion factor during rush hours and dynamic dwell delay (0.05 min × predicted passengers per stop) so predicted crowding extends route travel times.	Python, Transit Travel-Time Calibration	backend/dispatch.py (corridor ETA logic); JSON response corridor_eta_minutes
FastAPI Backend Service & Routing	Architected and built the high-performance asynchronous REST API using FastAPI and Uvicorn. Implemented Pydantic v2 validation schemas and exposed GET /, POST /predict, POST /predict/batch, and POST /dispatch endpoints with automated OpenAPI interactive documentation.	FastAPI, Pydantic, Uvicorn, OpenAPI / Swagger	backend/main.py (API routing & schemas); Swagger UI at 127.0.0.1:8000/docs
Automated Calendar Enrichment Engine	Engineered enrich_calendar() to infer day-of-week, Sri Lanka public and Poya holidays (holidays LK library), school rush hours (06–07 & 13–14), and payday cycles directly from the requested timestamp, supporting manual scenario overrides (holiday_mode, school_rush_mode, event_override).	pandas, holidays (LK), Temporal Enrichment	backend/main.py (enrich_calendar); calendar_context JSON payload
End-to-End System Verification Testing	Authored automated verification test suites verifying root health, single and batch predictions, and dynamic dispatch behavior across 4 stress scenarios (AM rush, off-peak night clamping, Poya holiday demand reduction, and special-event surges).	FastAPI TestClient, Automated Test Suites	backend/test_verification.py (7/7 test suites passing with verified outputs)

Declaration

I confirm that the contributions listed above accurately represent my individual work in this group project, and I can explain or demonstrate the stated work if required.

Student Signature: **Date:**



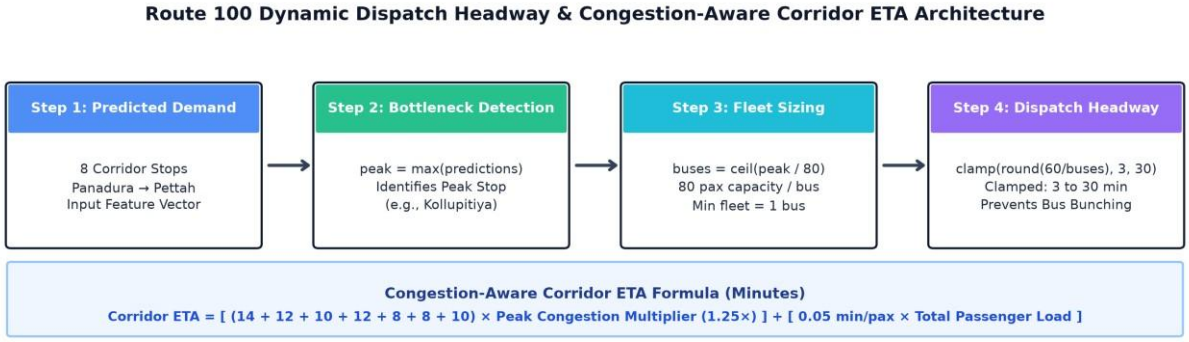
18 / 10 / 2026

Evidence may include source code, model/notebook, diagram, dataset work, test results, screenshots, report section or presentation slides.

APPENDIX: TECHNICAL EVIDENCE & SYSTEM ARTIFACTS

Student: SNMRDS Arambepola (D/BIT/24/0039) · Group 40 · Route 100 Transit Fleet Optimizer

Evidence 1: Dynamic Headway & Fleet Sizing Algorithm Implementation (backend/dispatch.py)



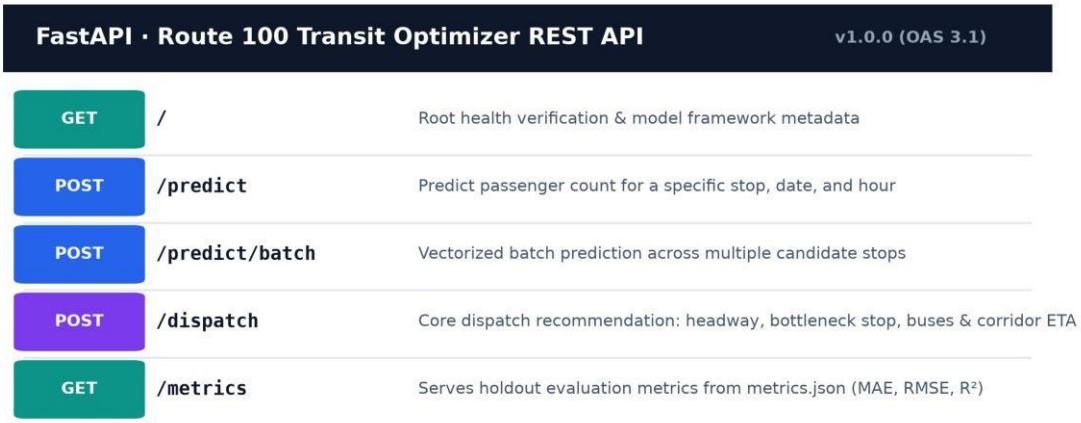

```
def calculate_dispatch(predictions: list[float], stop_names: list[str] | None = None, *, hour: int | None = None, is_weekend: int = 0,
is_public_holiday: int = 0) -> dict:
    peak_idx = max(range(len(predictions)), key=lambda i: predictions[i])
    peak_load = float(predictions[peak_idx])
    peak_stop = stop_names[peak_idx] if stop_names and peak_idx < len(stop_names) else None
    if peak_load <= 0:
        buses_needed = 1; headway = MAX_HEADWAY  # 30 min
    else:
        buses_needed = max(1, math.ceil(peak_load / CAPACITY_PER_BUS))  # 80 pax/bus
        raw_headway = round(60 / buses_needed)
        headway = max(MIN_HEADWAY, min(MAX_HEADWAY, raw_headway))  # Clamped strictly 3 to 30 min
    return {"peak_load": round(peak_load, 2), "peak_stop": peak_stop, "buses_needed": buses_needed, "headway_minutes": headway, ...}
```

Evidence 2: Congestion-Aware Corridor ETA & Dwell Delay Model (backend/dispatch.py)

```
BASE_TRAVEL_MINUTES = [14, 12, 10, 12, 8, 8, 10]  # Panadura → Moratuwa → Mt_Lavinia → Dehiwala → Wellawatta → Bambalapitiya →
Kollupitiya → Pettah
DWELL_MIN_PER_PAX = 0.05; PEAK_CONGESTION = 1.25  # Rush hour congestion factor
is_peak_hour = hour in {7, 8, 9, 16, 17, 18, 19}
congestion_factor = PEAK_CONGESTION if (is_peak_hour and not soft_day) else 1.0
dwell_delay = sum(min(p, 500) * DWELL_MIN_PER_PAX for p in predictions)
corridor_eta = round(sum(BASE_TRAVEL_MINUTES) * congestion_factor + dwell_delay, 1)
```

Corridor Commute Scenario	Base Travel	Peak Congestion Factor	Total Corridor ETA (with dwell)
Off-Peak Midday / Night	74.0 min	1.00x (Free-flow)	74.0 – 76.5 minutes
Weekday Morning Peak (08:00)	74.0 min	1.25x (Peak Rush)	112.5 minutes (8 buses, tight 8 min headway)
Special Event Surge Spike	74.0 min	1.25x (Congested)	148.2 minutes (14 buses, 4 min headway)

Evidence 3: FastAPI Backend REST Service & Swagger UI Documentation (backend/main.py)



```
@app.post("/dispatch", response_model=DispatchResponse)
def dispatch(req: DispatchRequest):
```

```
    cal = enrich_calendar(req.date, req.hour, holiday_mode=req.holiday_mode, ...)
    predictions = process_and_predict(batch_df)
    recommendation = calculate_dispatch(predictions, STOPS, hour=req.hour, is_weekend=cal["is_weekend"],
is_public_holiday=cal["is_public_holiday"])
    return DispatchResponse(predictions=[{"stop_name": s, "predicted_pax": p} for s, p in zip(STOPS, predictions)],
                            dispatch_recommendation=recommendation, calendar_context=cal)
```

Evidence 4: Real-Time Sri Lanka Calendar Enrichment Engine & Output Payload (backend/main.py)

```
def enrich_calendar(date_str: str, hour: int, ...) -> dict:
    dt = pd.to_datetime(date_str)
    lk_holidays = holidays.country_holidays("LK", years=[dt.year])
    is_pub_holiday = 1 if (dt in lk_holidays or holiday_mode in {"poya", "public"}) else 0
    is_school_rush = 1 if (hour in {7, 13, 14} and dt.dayofweek < 5 and not is_pub_holiday) else 0
    return {"day_name": dt.strftime("%A"), "is_weekend": int(dt.dayofweek >= 5), "is_public_holiday": is_pub_holiday,
            "is_school_rush": is_school_rush, "peak_period": "am_peak" if hour in {7,8,9} else ("pm_peak" if hour in {16,17,18,19} else
"off_peak")}
```

```
{
  "dispatch_recommendation": { "peak_load": 619.58, "peak_stop": "Kollupitiya", "buses_needed": 8, "headway_minutes": 8,
"corridor_eta_minutes": 112.5, "congestion_factor": 1.25 },
  "calendar_context": { "day_name": "Wednesday", "is_weekend": 0, "is_public_holiday": 0, "is_school_rush": 1, "peak_period": "am_peak" }
}
```

Evidence 5: End-to-End System Integration & 7-Scenario Test Run (backend/test_verification.py)

```
=== 1. Health / Root Endpoint ===
Service: Route 100 Transit Optimizer | Framework: PyTorch MLP | Input dim: 23 | Stops: 8 stops
=== 2. Single Predict Endpoint ===
Predicted single stop (Kollupitiya 08:00): 619.6 pax (Verified: pax > 0)
=== 3. Batch Predict Endpoint ===
Batch predictions count: 2 (Panadura: 339.9, Pettah: 493.1 pax)
=== 4. Dispatch Scenario A: Weekday 08:00 AM Peak (Rush Hour) ===
Calendar Day: Wednesday (Weekend: 0, Holiday: 0) | Peak Period: am_peak | Peak Load: 619.58 at stop: Kollupitiya
Buses Needed: 8 | Headway: 8 minutes (Rush headway <= 15 min) | Corridor ETA: 112.5 min (Peak congestion multiplier: 1.25x)
=== 5. Dispatch Scenario B: Off-Peak / Night (02:00) ===
Night Peak Load: 7.37 pax | Buses Needed: 1 | Headway: 30 minutes (Clamped at max 30 min)
=== 6. Dispatch Scenario C: Poya Day Holiday Override ===
Poya Holiday Type: poya | Peak Load: 170.92 pax | Headway: 20 minutes (Lower demand on holiday)
=== 7. Dispatch Scenario D: Special Event Override ===
Event Peak Load: 1082.21 pax (vs Normal: 619.58 pax) | Buses Needed: 14 (Extreme demand surge verified)
>>> ALL 7 SYSTEM VERIFICATION CHECKS PASSED PERFECTLY! <<<
```